

Coral-Chords documentation

Linus Tibert

August 4, 2025

Contents

1	Used abbreviations	3
2	Introduction	4
3	Project structure	4
3.1	Coral-Chords-Backend	4
3.2	Coral-Chords	4
4	Getting the data	4
5	Storing the data	4
6	UI stuff	4
7	Code components	4
7.1	Coral-Chords-Backend	4
7.1.1	Constants	4
7.1.2	Structs	5
7.1.3	Tests	5
7.2	Coral-Chords	6
7.2.1	Structs	6

1 Used abbreviations

Although some of those might seem obvious, I'll give a complete overview of special abbreviations used in the document for those who are not familiar with any of them.

CCh	Coral-Chords
UG	Ultimate Guitar
BE	Backend (coral-chords-backend library)
FE	Frontend (coral-chords binary application)
UI	User-interface
ID/UID	(unique) identifier

2 Introduction

This document is supposed to be a useful guide for those who want to understand how the code of CCh works. It is especially designed to be an explanation of how each component of the program is working, however, some detailed parts (such as specific functions()) are not mentioned here, because they are easier to understand (if you know what the whole section of the code is for), when looking at the actual code.

To make this document more accessible for everyone, it is divided into many small sections, each with a clear subject. The

3 Project structure

The project is divided into two major parts: the BA (coral-chords-backend library), and the main program (coral-chords binary; the FE).

The FE is the part of the program which interacts directly with the user and which is also creating the UI.

The BA is where the magic happens. Its purpose is to do anything not directly involving the user. Thus, it fulfils tasks involving networking, data extraction or direct interaction with the (file) system.

Dividing CCh this way makes it easier to focus on a specific task when extending the code. This way, a specific function in the BA can be heavily modified without affecting the UI. Another important aspect of this structure is the length of each part being dramatically reduced, helping to understand specific parts of code without having too much other stuff in the way.

3.1 Coral-Chords-Backend

3.2 Coral-Chords

4 Getting the data

5 Storing the data

6 UI stuff

7 Code components

7.1 Coral-Chords-Backend

7.1.1 Constants

END_OF_CHORDS_DELIM is a short but unique piece of code from the UG source HTML, which is used to mark the end of the section containing the chords and lyrics.

`START_OF_CHORDS_DELIM` the same as `END_OF_CHORDS_DELIM`, but used to mark the beginning of useable data. Note that the metadata is not included between the delimiters; it is above the start delimiter.

`HTML_BLACKLIST` is an array of strings which identify certain types of UG pages which are not downloadable; video pages for example.

`DETAIL_REGEX` is the Regex expression used to get detailed metadata from the raw HTML of a supported UG page. Each capturing group holds one property.

`TYPE_REGEX` is used to extract the type of supported page. (I.e. guitar chords, tabs, ukulele chords, bass tabs or drum tabs) This data is in the second capturing group. The first group extracts the artist's name, which is not used in the code yet.

7.1.2 Structs

`DataLine` is used to store a single line of extracted data together with the type of data stored on the line. Those lines are used in the `SongData` struct.

`SongData` is used to store a single song with all its collected data. The `lines` are the same lines which are written and loaded in local files.

`BasicSongData` holds the basic metadata every song has. It consists of the song title, artist, and the Spotify song UID.

`SongMetadata` is used to store optional metadata about a song. It is able to hold the capo position, tonality, tuning name and tuning details. Some of this data is not available on every supported page; in such cases, the unavailable data will be an empty string. If no metadata is available at all, `SongMetadata::default()` will be used as a metadata struct.

7.1.3 Tests

The library contains a bunch of tests to verify the code is still compatible with the currently available chords on UG.

The tests are named with what they are testing for. For example, the `type-detection()` test compares the results of detecting the types of a bunch of URLs using `get_type()` with a pre-defined type.

All tests are designed to work with as many types of pages as possible. This doesn't only include types like "Bass", "Guitar", etc., but also special pages of the same type (e.g., pages with more or less metadata than others).

If you extend the code in any way, make sure, all tests are passing before you submit a pull request. If you add support for any other type of page than currently supported, make sure to extend all tests appropriately.

7.2 Coral-Chords

7.2.1 Structs

N.b. this part of code uses many of the structs defined in the BE-part of the code.
View section *7.1.2 Structs* for information about the structs defined there.